A Short Note on Surprisal, Entropy, and Cross-Entropy Loss [Draft 1]

Devi Prasad M
Manipal School of Information Sciences
October 2026

1 Surprisal of an outcome

Let $\mathbf{q}$ be a discrete probability distribution over possible outcomes, with $q_k = \mathbf{q}(X = k)$. The *surprisal* of outcome k under $\mathbf{q}$ is

$$h_{\mathbf{q}}(k) = \log \frac{1}{q_k} = -\log q_k. \tag{1}$$

If $q_k = 0$, we define $h_{\mathbf{q}}(k) = +\infty$. This is a quantity for *one specified outcome*. We will see later that entropy is the average surprisal over outcomes.

Throughout this note, log means the natural logarithm ln, so surprisal is measured in nats.

Why the minus sign? Let us first look at $\log q_k$ on its own. Figure 1 shows it for $q_k \in (0, 1]$.

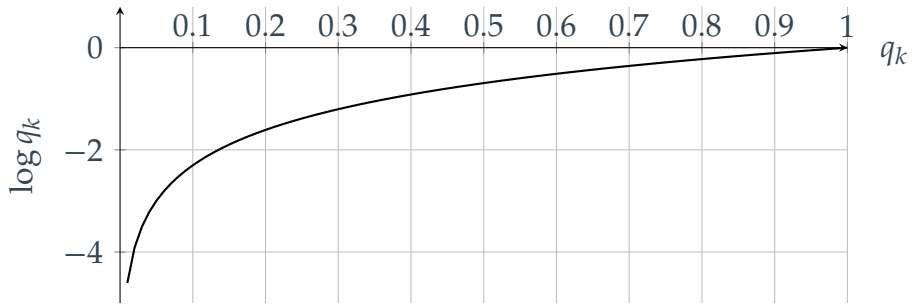


Figure 1: The curve $\log q_k$ for $q_k \in (0, 1]$. It is 0 at $q_k = 1$, is negative everywhere else, and drops without bound as $q_k \to 0^+$.

Two things are clear from the curve.

- **Direction:** $\log q_k$ goes *up* as q_k goes up. But surprise should go the other way: a likely outcome should be unsurprising, and a rare one very surprising.
- **Sign:** $\log q_k \leq 0$. But we want surprise to be a non-negative number, like a length.

Putting a minus sign in front flips the curve about the horizontal axis, and this fixes both problems:

- $q_k = 1$ gives surprisal 0. A certain outcome is not surprising at all.
- $q_k \to 0^+$ gives surprisal $\to \infty$. A nearly impossible outcome is very surprising.

Figure 2 shows the flipped curve.

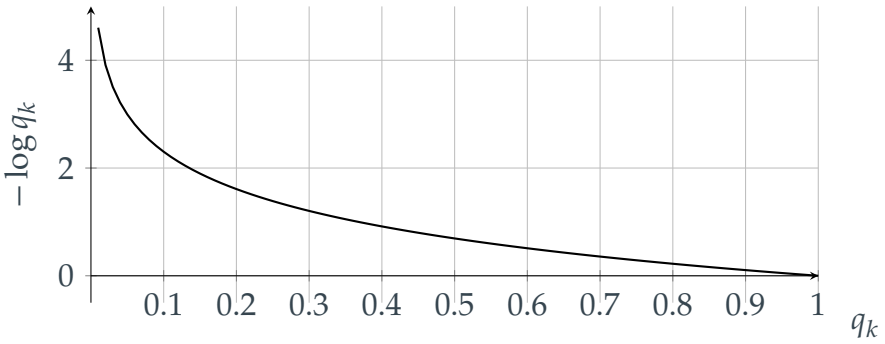


Figure 2: Surprisal $-\log q_k$ against q_k. It is 0 at $q_k = 1$ and grows without bound as $q_k \to 0^+$.

2 Why surprisals add

For two independent events A and B, probabilities multiply:

$$P(A, B) = P(A)\,P(B). \tag{2}$$

We want a measure of surprise that *adds* for independent events, just like length or mass. The logarithm turns a product into a sum. Writing $h(A) = -\log P(A)$ for the surprisal of an event, we get

$$\begin{aligned} h(A, B) &= -\log P(A, B) = -\log\left(P(A)\,P(B)\right) \\ &= -\log P(A) - \log P(B) = h(A) + h(B). \end{aligned} \tag{3}$$

Thus, adding surprisals is nothing but the multiplication rule of probability, transformed via a logarithm. This is the same reason we use log-likelihood for i.i.d. data, turning a product of probabilities into a sum.

3 Entropy: average surprisal

If we draw N independent outcomes from $\mathbf{q}$, outcome k comes up about Nq_k times, and each time it brings surprisal $-\log q_k$. The average surprisal per draw is therefore

$$H(\mathbf{q}) = \sum_k q_k\,(-\log q_k) = -\sum_k q_k \log q_k. \tag{4}$$

The factor q_k is simply the fraction of time outcome k actually happens. Every average needs such weights.

Here $\mathbf{q}$ plays two roles: it gives the *weights*, and it also decides the *surprisal*.

Outcomes with $q_k = 0$ contribute nothing. The term $q_k(-\log q_k)$ is then of the form $0 \cdot \infty$, which we take to be 0 by convention ($0 \log 0 = 0$, justified because $q \log q \to 0$ as $q \to 0^+$).

Example. Take $\mathbf{q} = (0.9, 0.1)$. Then

$$H(\mathbf{q}) = 0.9 \times 0.105 + 0.1 \times 2.303 \approx 0.325 \text{ nats.} \tag{5}$$

The common outcome has small surprisal and large weight, so it dominates the average.

Two observations.

1. If one outcome has probability 1, then $H = 0$.
2. If all K outcomes are equally likely, then $H = \log K$, the maximum.

$$H(\mathbf{q}) = \sum_{k=1}^{K} q_k \,(-\log q_k) = \sum_{k=1}^{K} \frac{1}{K}\Big(-\log \frac{1}{K}\Big) = K \cdot \frac{1}{K} \cdot \log K = \log K. \tag{6}$$

4 Cross-entropy: separating the two roles

Now suppose outcomes really occur according to a distribution $\mathbf{p}$ (the truth), but we measure surprise using our model $\mathbf{q}$. Then the weights come from $\mathbf{p}$ and the surprisal comes from $\mathbf{q}$:

$$H(\mathbf{p}, \mathbf{q}) = \sum_{k} p_k \,(-\log q_k) = -\sum_{k} p_k \log q_k. \tag{7}$$

In simple words: cross-entropy is the average surprise of the model $\mathbf{q}$ when the data actually comes from $\mathbf{p}$.

Replacing $\mathbf{q} = \mathbf{p}$ gives back the entropy:

$$H(\mathbf{p}, \mathbf{p}) = -\sum_{k} p_k \log p_k = H(\mathbf{p}). \tag{8}$$

5 One-hot labels

In classification, the label is one-hot. If the true class is c, then $p_c = 1$ and $p_k = 0$ for all $k \neq c$. Every term with $p_k = 0$ vanishes, and Eq. (7) leaves us with

$$H(\mathbf{p}, \mathbf{q}) = -\log q_c. \tag{9}$$

So the cross-entropy loss for one example is simply *how surprised the model is by the correct class*. If q_c is near 1, the loss is near 0. If q_c is near 0, the loss is large.

The probabilities given to wrong classes do not appear directly. They matter only because $\mathbf{q}$ sums to 1: whatever mass goes to wrong classes is taken away from q_c.

6 Surprisal, likelihood, and log-likelihood

We often come across these five terms: surprisal, likelihood, log-likelihood, entropy, and cross-entropy. All these ideas are directly related to one another. In this section we will connect them.

A data set of N examples. In training, we have N examples $(x_1, c_1), \ldots, (x_N, c_N)$, where c_i is the true class of input x_i. Let θ denote the parameters of the network. We write $q_\theta(c_i \mid x_i)$ for the probability that the network gives to the correct class of example i. This is just the q_c of Eq. (9), written for example i.

Likelihood. Assume the examples are independent. Then, by Eq. (2), the probability that the network gives to *all* the observed labels together is a product:

$$P(c_1, \ldots, c_N \mid x_1, \ldots, x_N;\ \theta) = \prod_{i=1}^{N} q_\theta(c_i \mid x_i). \tag{10}$$

When we keep the data fixed and look at this number as a function of θ, we call it the *likelihood* of θ. A good θ is one that makes the observed labels probable, so we want the likelihood to be large.

Log-likelihood. The logarithm turns this product into a sum, just as in Eq. (3):

$$\ell(\theta) = \log \prod_{i=1}^{N} q_\theta(c_i \mid x_i) = \sum_{i=1}^{N} \log q_\theta(c_i \mid x_i). \tag{11}$$

This is the *log-likelihood*. Each term is ≤ 0, so $\ell(\theta) \leq 0$. Since log is an increasing function, making $\ell(\theta)$ large is the same as making the likelihood large.

There is also a practical reason to prefer the log. A product of thousands of numbers below 1 becomes so small that a computer rounds it to 0. The sum in Eq. (11) has no such problem.

Negative log-likelihood is total surprisal. Now put a minus sign in front of Eq. (11). Each term becomes a surprisal, Eq. (1):

$$-\ell(\theta) = \sum_{i=1}^{N} \big(-\log q_\theta(c_i \mid x_i) \big) = \text{total surprisal of the observed labels.} \tag{12}$$

This quantity is called the *negative log-likelihood* (NLL). So surprisal and log-likelihood are the same numbers with opposite signs: log-likelihood measures how *plausible* the model finds the data, and surprisal measures how *surprised* the model is by it.

Average surprisal is the cross-entropy loss. Divide Eq. (12) by N:

$$\bar{L}(\theta) = -\frac{1}{N}\,\ell(\theta) = \frac{1}{N}\sum_{i=1}^{N}\big(-\log q_\theta(c_i \mid x_i)\big). \tag{13}$$

Each term in the sum is the one-hot cross-entropy of Eq. (9) for one example. So $\bar{L}(\theta)$ is the average cross-entropy loss over the data set, which is exactly the number that deep learning libraries report.

Putting it all together, these four statements are the same:

$$\begin{aligned} & \text{maximise the likelihood} \\ \iff\ & \text{maximise } \ell(\theta) \\ \iff\ & \text{minimise the total surprisal } -\ell(\theta) \\ \iff\ & \text{minimise } \bar{L}(\theta). \end{aligned}$$

The first step holds because log is increasing. The second is just a change of sign. The third holds because dividing by N does not change which θ is best. This method of choosing θ is called *maximum likelihood estimation*.

Example. Take a coin model with $\mathbf{q} = (0.6, 0.4)$ for (Heads, Tails), and suppose we observe Heads, Heads, Tails. Then

$$\begin{aligned} \text{likelihood} &= 0.6 \times 0.6 \times 0.4 = 0.144, \\ \text{log-likelihood} &= \log 0.144 \approx -1.938, \\ \text{total surprisal} &= 0.511 + 0.511 + 0.916 \approx 1.938, \\ \text{average surprisal} &\approx 1.938/3 \approx 0.646 \text{ nats}. \end{aligned}$$

The same number appears in every line, only with a change of sign or a division by $N = 3$.

Who is who. Table 1 puts the five names side by side.

Name	What it is about	Needs observed data?	In training
Surprisal	one outcome	no	a score
Entropy	one distribution	no	a property
Likelihood	the model's θ	yes	make large
Log-likelihood	the model's θ	yes	make large
Cross-entropy loss	the model's θ	yes	make small

Table 1: Five names and how they relate. Negative log-likelihood is total surprisal, and dividing it by N gives the average cross-entropy loss.

Two points to keep in mind.

1. The likelihood is a function of θ, with the data held fixed. It is *not* a probability distribution over θ: its values over different θ need not add up to 1.

2. Entropy $H(\mathbf{q})$ belongs to a single distribution, and we can compute it without seeing any data. Likelihood, log-likelihood, and the cross-entropy loss always involve observed outcomes: they ask how well the model explains what actually happened.

7 From logits to q: the softmax

A network does not output probabilities directly. It outputs real numbers $z_1, \dots, z_K$, called *logits*. The softmax converts them into a distribution:

$$q_k = \frac{e^{z_k}}{\sum_j e^{z_j}}. \tag{14}$$

Each q_k is positive and they add up to 1, so $\mathbf{q}$ is a valid distribution.

Putting this in the loss for true class c:

$$L = -\log q_c = -z_c + \log \sum_j e^{z_j}. \tag{15}$$

In simple words: the loss is small when the correct logit z_c is large compared with the other logits. The log-sum-exp is then close to z_c, and the two terms in Eq. (15) nearly cancel. Raising z_c also raises the log-sum-exp, but by less (by q_c per unit), so the loss still falls.

Note. Deep learning libraries compute L directly from the logits using this last form (with the log-sum-exp trick), instead of first computing q_k and then taking its log. This avoids overflow in e^{z_j} and avoids $\log 0$.

Gradient. Differentiating with respect to a logit gives

$$\frac{\partial L}{\partial z_k} = q_k - p_k. \tag{16}$$

This is the model's probability minus the label. Gradient descent moves each logit against its gradient. For the true class the gradient is $q_c - 1 \leq 0$, so the logit is pushed up. For the wrong classes it is $q_k \geq 0$, so those logits are pushed down.

8 Summary

Eq.	Quantity	Meaning
(1)	$-\log q_k$	surprisal of outcome k under $\mathbf{q}$
(4)	$H(\mathbf{q}) = -\sum_k q_k \log q_k$	average surprisal under $\mathbf{q}$
(7)	$H(\mathbf{p}, \mathbf{q}) = -\sum_k p_k \log q_k$	average surprisal under $\mathbf{q}$, data from $\mathbf{p}$
(9)	$-\log q_c$	cross-entropy loss for one-hot label c
(11)	$\ell(\theta) = \sum_i \log q_\theta(c_i \mid x_i)$	log-likelihood of the data set
(13)	$\bar{L}(\theta) = -\ell(\theta)/N$	average cross-entropy loss